

Markov Chains: Concept and Applications in Data Science

Markov chains are mathematical models of first-order stochastic processes, characterized by the short-memory property: the future state of a system depends solely on its present state, and not on the complete sequence of previous states. In other words, they fulfill the Markov property, whereby given the current state, the forecast for the next instant does not need additional information from the distant past. This idea, introduced by the Russian mathematician A.A. Markov in 1906, revolutionized the way of modeling time-dependent processes. Since then, Markov chains have become a fundamental tool in numerous fields, from statistical physics to modern artificial intelligence. Far from being a merely theoretical concept, today they are a key methodological pillar for extracting value from sequential data in the digital economy. Below is a detailed explanation of what Markov chains are, their mathematical formulation, illustrative examples, and various practical applications in data science, with an emphasis on the calculation of numerical probabilities.

Definition and Markov Property

A Markov chain is a type of discrete stochastic process in which the distribution of the next state depends only on the current state. Formally, a sequence of random variables $\{X_n\}_{n \geq 0}$ with values in a set of states E is a Markov chain if for all n and for all states $i_1, \dots, i_n, i_{n+1} \in E$, the first-order relation (Markov property) is fulfilled:

$$P(X_{n+1} = i_{n+1} \mid X_1 = i_1, \dots, X_n = i_n) = P(X_{n+1} = i_{n+1} \mid X_n = i_n), \tag{1}$$

which indicates that the probability of transition to the future state i_{n+1} depends only on the present state i_n and not on the complete sequence of past states. In intuitive terms, “the immediate future of a Markov process depends on the present, not the past.” When we can directly observe the sequence of states $\{X_n\}$ at each instant, we speak of an observable Markov model; if the real states are not directly observable (only some indirect signals are), it is a Hidden Markov Model (HMM). In this report, we will focus on observable Markov chains, unless otherwise indicated.

Mathematical Representation and Fundamental Elements

Mathematically, a discrete-time Markov chain is specified by the following key elements:

- **State Space (E):** A finite (or countable) set of all possible states the system can take. For example, in a simple weather model $E = \{\text{sun}, \text{clouds}, \text{rain}\}$. Each state represents a relevant situation or category of the system being modeled.
- **Transition Probabilities:** For each pair of states $i, j \in E$, $p_{ij} = P(X_{n+1}=j \mid X_n=i)$ is defined, which is the probability of transitioning from state i to state j in one time step. These probabilities are usually assumed to be homogeneous in time, that is, constant at each stage (they do not depend on n).
- **Initial State or Initial Distribution:** The chain requires a starting condition. It can be a fixed initial state $X_0 = i_0$ (e.g., “today is rainy”) or an initial probability distribution

$\boldsymbol{\pi}^{(0)}$ indicating the probability of starting in each state. This initial vector evolves according to the transitions to give probabilities in future steps.

The set of transition probabilities is usually represented in the form of a **transition matrix** $P = (p_{ij})$. This is a square matrix of dimension $|E| \times |E|$ where the entry p_{ij} corresponds to the probability of moving from state i to state j in a single step. For example, if $E = \{\text{sun}, \text{clouds}, \text{rain}\}$, a possible transition matrix could be:

Current State → Next State ↓	Sun	Clouds	Rain
Sun	0.35	0.25	0.40
Clouds	0.40	0.25	0.35
Rain	0.35	0.45	0.20

In this example matrix P , each row corresponds to a current state and distributes probabilities among the possible transitions to future states. By construction, every transition matrix must fulfill two essential properties: (1) all its entries are non-negative numbers ($p_{ij} \geq 0$), and (2) the sum of each row is 1 ($\sum_j p_{ij} = 1$ for each state i). The first condition guarantees that the probabilities are valid (there cannot be negative probabilities), and the second reflects that from any state the system must transition to *some* state in the next step (it is even possible to remain in the same state, which also counts as a transition). In the example matrix above, for instance, the row for the state “Sun” sums to $0.35 + 0.25 + 0.40 = 1$, indicating that if today is sunny, tomorrow it will be sunny, cloudy, or rainy with 35%, 25%, and 40% probability respectively.

Homogeneity Property: In many case studies, we assume that the transition matrix P is constant over time (homogeneous processes). However, it is worth mentioning that there are non-homogeneous Markov chains where P can change at different stages (for example, seasonal changes in a climate model). In this report, we will focus on the most common homogeneous case, in which p_{ij} does not depend on time.

Illustrative Example: Markov Chain for Weather Prediction

Figure 1: Diagram of states and transitions for a simplified meteorological model. The possible states are Sun, Clouds, and Rain. The directed arrows indicate the transition probabilities from one day to the next; for example, if today is sunny, there is a 40% probability of rain tomorrow, 25% probability of clouds, and 35% that it remains sunny (arrows exiting the Sun state).

Similarly, from Rain, it transitions to Sun with 35% probability, to Clouds with 45%, or remains Rain with 20%. Each state has an associated transition probability distribution whose sum is 1.

Suppose we have determined the transition matrix for daily weather based on historical observations (like the one shown in the figure). If day 0 begins in the state Rain, we can use this matrix to predict the probabilities for day 1 (tomorrow) and subsequent days. For example, starting from rain today, the probability distribution for tomorrow will be: 35% sun, 45% clouds, 20% rain (as indicated in the “Rain” row of the matrix). To obtain these probabilities, we take the row corresponding to the current state (“Rain”) in matrix P . Continuing with the calculation, the probability that the day after tomorrow is sunny can be estimated by considering all possible routes in two steps: rain $\rightarrow$ sun or rain $\rightarrow$ clouds $\rightarrow$ sun, etc. Instead of enumerating each path manually, we use the mathematical property of Markov chains: the distribution of states at n steps seen from today is obtained by raising the transition matrix to the power of n . Thus, the entry $(P^2)_{\text{rain, sun}}$ of the matrix P^2 (transition matrix in two steps) gives us directly the probability of going from rain today to sun in two days. In this example, $(P^2)_{\text{rain, sun}} \approx 0.37$ (37%), indicating that there is approximately a 37% probability that the day after tomorrow is sunny given that it rains today.

In addition to calculating marginal distributions, we can also compute the probability of a specific sequence of states day after day. Thanks to the Markov property, the probability of a particular trajectory is easily obtained as the product of the transition probabilities at each step. For example, resuming the previous scenario (today it rains), the probability that the next five days follow the exact sequence Sun $\rightarrow$ Sun $\rightarrow$ Rain $\rightarrow$ Clouds $\rightarrow$ Sun would be:

- Day 1: from rain to sun (probability 0.35),
- Day 2: from sun to sun (probability 0.35),
- Day 3: from sun to rain (probability 0.40),
- Day 4: from rain to clouds (probability 0.45),
- Day 5: from clouds to sun (probability 0.40).

Multiplying these values step by step: $0.35 \times 0.35 \times 0.40 \times 0.45 \times 0.40$, we obtain the joint probability (a small number, less than 1%). In general, denoting a sequence of states i_0 to i_1 to $\dots$ to i_n , it holds that:

$$P(X_0=i_0, X_1=i_1, \dots, X_n=i_n) = P(X_0=i_0) \cdot p_{i_0 i_1} \cdot p_{i_1 i_2} \cdot \dots \cdot p_{i_{n-1} i_n},$$

where $P(X_0=i_0)$ is the initial probability of starting at i_0 (which is usually 1 if i_0 is a fixed initial state). This calculation is simple because the first-order structure avoids having to consider cases dependent on states further back: each factor $p_{i_k i_{k+1}}$ reflects that, given state i_k at step k , the probability of the next i_{k+1} is already independent of any state prior to i_k .

Note: When repeating the process of multiplying the distribution by the transition matrix (or raising P to successive powers), sometimes the probabilities tend to stabilize. Under certain theoretical conditions (irreducible and aperiodic chains), there exists a **stationary distribution**

$\boldsymbol{\pi}$ such that $\boldsymbol{\pi} = \boldsymbol{\pi} P$. This limit distribution describes the fraction of time the system spends in each state in the long run, regardless of the initial state. In the weather example, if the chain were ergodic, after many days the probability of sun, clouds, or rain would converge to constant values (e.g., 40%, 30%, 30% each) that no longer change day by day. This idea of equilibrium is key in applications like the PageRank algorithm, which we will see later.

Calculation of Probabilities and Numerical Predictions

One of the main practical utilities of Markov chains is the prediction of future events and the calculation of associated probabilities, starting from a known transition matrix and an initial state (or distribution). As mentioned, if we represent by $\boldsymbol{\pi}^{(0)}$ the row vector of initial probabilities (distribution over E at time 0), then the distribution of states after n steps is given by:

$$\boldsymbol{\pi}^{(n)} = \boldsymbol{\pi}^{(0)} P^n,$$

where $(P^n)_{ij} = P(X_n = j \mid X_0 = i)$ is the probability of going from state i to state j in exactly n steps. In particular, the i -th row of P^n provides the probability distribution of the chain at step n conditioned on having started in state i . If i was the initial state, that row is precisely the sought distribution. For example, in the previous weather model, if we want to know the probability that in 10 days it will be sunny, cloudy, or rainy (regardless of the intermediate state of each day), we can calculate the matrix P^{10} ; the row corresponding to today's state would directly give us those probabilities for day 10. This allows making long-term forecasts efficiently. In fact, with P^{10} the weather model would provide us with the weather probabilities 10 days out, always under the assumption that the transition patterns remain constant.

Another way to see this calculation is through iterative multiplication: to obtain $\boldsymbol{\pi}^{(n)}$, we can successively multiply $\boldsymbol{\pi}^{(0)}$ by P (for 1 step), then the result by P again (for 2 steps), and so on up to n times. In each multiplication, the distribution “advances” one step in time. Each element of $\boldsymbol{\pi}^{(n)}$ is in turn a sum of products of the previous distribution with the transition probabilities, reflecting all possible routes of length n that lead to a given state.

Numerical Example: Let's take the transition matrix P from the climate example and suppose again that today the state is Rain (initial distribution $\boldsymbol{\pi}^{(0)} = (0, 0, 1)$ concentrated on Rain). Then:

- **After 1 step (tomorrow):** $\boldsymbol{\pi}^{(1)} = \boldsymbol{\pi}^{(0)} P = (0, 0, 1)P = (0.35, 0.45, 0.20)$, that is 35% Sun, 45% Clouds, 20% Rain (coinciding with the Rain row in P).
- **After 2 steps (day after tomorrow):** $\boldsymbol{\pi}^{(2)} = \boldsymbol{\pi}^{(1)} P = (0.35, 0.45, 0.20)P \approx (0.3725, 0.2900, 0.3375)$. In other words, there is $\approx 37.3\%$ probability of Sun, 29.0% of Clouds, and 33.8% of Rain after two days.
- **After 3 steps:** $\boldsymbol{\pi}^{(3)} = \boldsymbol{\pi}^{(2)} P \approx (0.3645, 0.3175, 0.3180)$. We see that the probabilities begin to stabilize.

We can observe in this calculation that the probability of sun first increased from 35% to ~37%, and then tends to stabilize around ~36%. If we continued multiplying, the distribution would gradually approach a limit value $\boldsymbol{\pi}$ (in this case close to $(0.365, 0.315, 0.320)$), which corresponds to the mentioned stationary distribution. In the long run, the effect of the initial conditions is “forgotten,” and the probabilities converge to that equilibrium (if the chain is ergodic). This behavior is mathematically guaranteed by the ergodic theorem under certain conditions, and is exploited in many practical applications.

In summary, the calculation of probabilities with Markov chains is summarized as follows: starting from a known initial distribution (or a specific initial state), we can obtain future distributions through the power of the transition matrix, and we can calculate probabilities of sequences through the product of probabilities step by step. These operations allow everything from estimating the probability of an event n steps ahead to determining the most probable sequence of events in a certain horizon, given the Markovian model.

Practical Applications of Markov Chains in Data Science

Thanks to their simplicity and predictive power, Markov chains are widely used in data science to model sequential phenomena and make probabilistic predictions. Below, several prominent applications are described, emphasizing how the Markovian structure is leveraged in each case.

Prediction of Future Events and Forecasts

Markov models are very useful for predicting future events in systems where the state evolves over time with uncertainty. A classic example is simplified weather prediction: assuming that daily weather follows a Markov chain (as in our illustrative example), we can project the probability of each weather type in coming days using the transition matrix. For example, calculating P^n directly gives us the probabilities n days out, which serves to obtain probabilistic forecasts (e.g., “there is a 40% probability of rain in 3 days”).

In industrial and IoT (Internet of Things) environments, this approach applies to **predictive maintenance**: sensors installed on machinery generate sequences of states (operation measurements, alerts, etc.), and through Markov chains, probable failures or breakdowns in the near future are anticipated. Each state could represent a level of operation or equipment wear, and the transitions model the probability that it worsens or fails in the next interval. This allows estimating, for example, the probability that a machine breaks down in the next week given its current state, and thus planning maintenance before a critical failure occurs. Similarly, Markov chains can be used to **detect anomalies** in time series: if a certain sequential pattern of measurements has a very low probability under the learned Markovian model (that is, it is an atypical sequence), it could indicate an anomalous event or malfunction.

In reliability and lifespan, Markov chains model the transition of components through states like “functioning,” “degraded,” “failed.” Thus it is possible to calculate metrics such as the probability of survival until a certain time or the mean time to failure, directly integrating the transition probabilities between health states of the component. These numerical predictions help make informed decisions in equipment maintenance, energy demand forecasting (for example,

modeling electricity consumption patterns with different load states), and generally in any domain where it is of interest to estimate probabilities of future events in a sequential process.

Modeling Sequential Behavior (Web Navigation and Customer Flows)

Another area of great relevance is the analysis of sequential user behaviors, such as navigation through web pages or so-called *customer journeys* in digital marketing. Here, Markov chains allow going beyond the typical linear conversion funnel, incorporating the possibility of non-linear routes and multiple interactions. On a website, for example, each page or section can be considered a state, and user movements (clicks or transitions from one page to another) are the transitions of the chain. In this way, the path an average user follows when exploring the site is mathematically modeled.

Unlike a traditional funnel model (where we assume a fixed sequence of steps toward conversion), the Markov chain considers *all* possible routes that users could follow, better reflecting the reality of web navigation which is usually non-linear. By using Markov chains to generalize the conversion funnel, a more complete view of user behavior is obtained: it is possible to identify alternative navigation patterns, understand how users explore different sections of the site, and discover at which detours or pages they tend to abandon the process. For example, one can calculate the probability that a user who is currently on the home page ends up making a purchase (reaching the confirmation page) versus the probability that they abandon, considering all intermediate paths they could take. It is also feasible to determine the most probable next action of a user given their current state: e.g., “a user on the product detail page has a 20% probability of adding the product to the cart, a 15% probability of going to another category, a 5% probability of going to help, and a 60% probability of leaving the site.”

This type of sequential analysis is very valuable in digital marketing and web analytics. Some specific applications include:

- **Customer Journey Analysis:** Through Markov chains, the most common paths (and also the unusual ones) toward conversion on an e-commerce site are identified, which helps detect at which step more users are lost. For example, it could be revealed that many users navigate from step A to C skipping B, or that a certain step X has a high churn rate (abandonment) compared to another product or segment. With that information, measures can be taken to optimize the experience at that specific point (improve the page, simplify the form, etc.).
- **Multi-Channel Marketing Attribution:** The conversion process can be modeled including different entry channels as states (email, social networks, organic search, ads, etc.), so that transitions reflect how a user typically passes from one channel to another before converting. The Markov-based attribution algorithm calculates conversion probabilities with and without certain channels, identifying which channels have the greatest impact on the final result. This provides a dynamic way to assign credit to marketing channels instead of static “last click” rules.
- **Experience Optimization and Sequential A/B Testing:** Since Markov chains capture navigation dynamics, they serve to measure how a change on the site alters user routes. For example, with sequential A/B testing, a page variant can be implemented and the

transition probabilities in the user chain exposed to that variant versus the original can be observed. If the variant reduces the probability of abandonment in a certain state or increases the probability of advancing to the next step, the Markov model will reflect it quantitatively.

In summary, Markovian modeling of user behavior provides metrics such as transition probabilities between customer actions, abandonment rates from each state, and conversion probabilities from different starting points. These metrics exceed global conversion rates in granularity, allowing analysts to diagnose specific bottlenecks in the user experience and optimize interaction flows in an informed manner.

The PageRank Algorithm and Search Engines

One of the most famous applications of Markov chains is Google's **PageRank** algorithm for ranking web pages. PageRank models the random navigation of a user through the internet via a Markov chain. In this model, each web page is represented as a state, and the hyperlinks between pages act as state transitions (a link from page A to B is interpreted as a probability of “jumping” from A to B when following a link). In a simplified way, imagine a user who starts on any page and at each step chooses at random one of the outgoing links to continue their navigation; this is equivalent to traversing a web graph through a **random walk** indefinitely.

Under this scheme, the stationary distribution of the Markov chain (that is, the limit probabilities of being on each page after many steps) determines the importance or score of each page. In essence, a page will have a high rank if the stationary probability of being in that state is high, which happens if many other pages (states) “point” to it with links, especially if those pages are themselves important. Google leverages this result by calculating the stationary vector $\boldsymbol{\pi}$ of the Markov chain of the web graph (with some modifications like damping factors), and uses the values of $\boldsymbol{\pi}$ as the famous PageRank of each page. Thus, the problem of classifying pages is reduced to a linear calculation on the transition matrix of a potentially gigantic graph, something feasible through iterative methods that essentially raise the matrix to large powers until convergence.

The success of PageRank demonstrated the power of Markov chains to synthesize global information from a directed graph. It is worth noting that convergence to a unique stationary distribution is guaranteed because, by introducing a small “teleportation” probability factor (random jumps to *any* page), the chain becomes irreducible and aperiodic (i.e., ergodic). In this way, the conditions for a unique attractor stationary distribution to exist are met. In practice, the PageRank calculation amounts to solving $\boldsymbol{\pi} = \boldsymbol{\pi}P$ (an eigenvector of the largest eigenvalue of P^T), which is efficiently approximated with power-type iterations.

In summary, PageRank is an elegant example of how a Markov chain can be used to quantify a measure of global importance in a complex system. Every time we perform a search, this Markovian idea is being leveraged in milliseconds to order the results according to the web's link structure.

Natural Language Processing (NLP) and Text Generation

In the early stages of automatic language processing, before the rise of deep neural networks, Markov chains played a fundamental role in language modeling. A Markov-based language model (for example, an n-gram model for words or characters) assumes that the next word depends only on the last k previous words (with $k=1$ being the case of a simple first-order Markov chain). These Markovian models were precursors to modern architectures like GPT, and allowed performing basic NLP tasks such as:

- **Automatic Text Generation:** By chaining transition probabilities between words (or letters), phrases can be generated that mimic a certain corpus. For example, a Markov text generator learns from a training text the probabilities $P(\text{word}_{t+1} \mid \text{word}_t)$ for consecutive occurrences. Then, starting with an initial word, it randomly chooses the next word according to those distributions, producing sentences that statistically resemble the original text. Although phrases generated by this method may lack long-term coherence, they usually maintain a certain local fluency, demonstrating that Markov chains capture short-range linguistic patterns.
- **Spell Checking and Anomaly Detection in Text:** A character Markov model can assign probabilities to sequences of letters. Letter sequences of very low probability (e.g., “xzqv”) could signal typos, while common sequences (“tion” in English) would have high probability. In this way, word processors could suggest corrections based on which sequences of letters or words do not accord with the learned probabilistic model.
- **Text Labeling and Classification:** In tasks like spam filtering, separate Markov models could be trained for different classes (e.g., a word chain model for spam emails and another for non-spam). Then, given a new message, the probability that it was generated by each model is evaluated, choosing the class whose Markov chain assigns it the highest probability.

Currently, more advanced techniques (Recurrent Neural Networks, Transformers, etc.) have surpassed Markov models in performance for NLP. However, there are still simple applications where the Markovian approach is useful. For example, basic chatbots and predictive keyboards use Markov chains to predict the next word given the current word or the last few words. Smartphone keyboards, when suggesting words, often rely on a simple model that considers the last two or three written words and proposes the most probable next one (which is a Markov approach of order 2 or 3). These models are fast and require few resources, appropriate for mobile devices.

Furthermore, Markov chains remain a conceptual component in modern methodologies: a **Hidden Markov Model (HMM)**, widely used in speech recognition and sequence labeling, combines an underlying Markov chain (which models the sequence of hidden states, like phonemes in speech) with another emission model of observations. HMMs have been crucial for tasks like speech recognition (decades before neural models), where the phonemes of a word are modeled with a hidden Markov chain, or the identification of genes in DNA sequences, where hidden states represent coding or non-coding regions in the genome. Although HMMs imply an additional level of complexity (states not directly observable), they are still based on

the Markovian premise for state change dynamics, and on probabilistic algorithms (like Viterbi or Forward-Backward) to infer useful information from observed sequences.

Other Notable Applications

The versatility of Markov chains makes them appear in a multitude of additional contexts within data science and related disciplines. By way of closing, we briefly mention some further applications:

- **Markov Chain Monte Carlo (MCMC) Methods:** In computational statistics, Markov chains have revolutionized the way of sampling complex distributions. The central idea of MCMC is to construct a Markov chain whose stationary distribution is the target distribution we want to simulate. By generating a sufficiently long run of that chain, the frequencies of visits to states (or regions of continuous space) follow the desired distribution, allowing the estimation of expectations and uncertainties. This principle underlies algorithms like Metropolis-Hastings and Gibbs Sampling, widely used for Bayesian inference when the posterior distribution of parameters is difficult to calculate directly. In other words, MCMC leverages that, by letting the chain evolve, the collected samples eventually come from the distribution we seek. This technique is fundamental in modern statistics, allowing analytically intractable problems to be solved through random simulation.
- **Insurance and Finance:** Insurers employ Markov chains to model bonus-malus systems in automobile insurance. In this case, the states can be the customer's bonus classes (based on their claims-free history), and transitions depend on whether they had accidents in the last period. Thus, the premium an insured must pay over time is calculated dynamically, updating it according to their movement between states (for example, an accident causes a transition to a worse bonus state, increasing the premium). In quantitative finance, Markov chains also model market regime changes (bull, bear) or states of the economy, and are incorporated into risk and valuation models.
- **Queueing Systems and Service Availability:** In queueing theory, the states of a Markov chain can represent the number of customers waiting in a system (bank, call center, web server, etc.). The transitions model the arrival of new customers or the service of one of those waiting, changing the number in the queue. This Markovian representation allows calculating performance metrics such as mean waiting time, expected queue length, the fraction of time the system is empty or busy, etc., important for optimizing resources and reducing waiting times. For example, in a telephone call center, a state "5" could indicate 5 calls on hold; with certain call arrival rates and service rates, the transition matrix of that birth-and-death process is constructed, and from there long-term probabilities of having n customers in queue are derived, helping to dimension the number of operators needed to maintain a desired service level.

In conclusion, Markov chains provide a unifying framework for analyzing time-dependent random processes quantitatively. Their concise mathematical formulation (transition matrices and matrix powers) translates into efficient algorithms for probability computation, which

explains their wide adoption. We have seen how this model appears in contexts as diverse as weather prediction, web analytics, search engines, language processing, and many others. The strength of Markov chains lies in capturing the essential nature of probabilistic sequential dynamics without requiring a complete history, making them tractable even for large systems. Of course, not all phenomena fit perfectly with the Markov assumption; but when it is a reasonable approximation, the resulting model is usually elegant and of great practical utility. Markov chains remain, in the era of big data, a current and powerful tool for understanding and predicting the world through sequential data.

References: The statements and examples presented in this report are based on recognized literature and sources on Markov chains and their applications. Specific citations have been included throughout the text (in the format [n†Lx-Ly]) to indicate the origin of key information, methods, and use cases described. These references point to works and materials where the reader can delve deeper into each aspect mentioned. In particular, rigorous theoretical explanations, discussions of mathematical properties, as well as analyses of contemporary applications in data science are highlighted, among others. Each of them contributes additional context and technical detail on Markov chains in action.